AETHERIA / SMARTHOMES

Smart Home Commerce & Management Platform

Complete feature guide and technical project report

A detailed account of the customer experience, administration tools, backend services, supporting files, and operating workflow.

REPORT BASIS	EDITION
Source files in the smarthomes folder	11 October 2026
Customer storefront + admin console + unified API	Code review edition

Prepared in an editorial report format for project review, demonstration, and handover. Features are described from the supplied source code. External services and runtime behaviour have not been independently tested.

01 / PROJECT DOCUMENTATION

How to read this report

Aetheria brings the sales and planning side of a smart home business into one project. A customer can explore automation products, save a shortlist, compare devices, request a quotation, and prepare an estimated home automation proposal. Staff work in a separate console to maintain the catalogue, track requests, and manage customer and website information.

The project has three main applications. Two React interfaces provide the customer and staff experiences. An Express backend stores shared records, publishes update events, prepares payment orders, and generates invoice emails. Firebase and browser storage also appear throughout the interfaces, making this a hybrid implementation rather than a single database design.

Scope and evidence

This report covers first-party source files, routes, components, data models, configuration, launch scripts, and project documentation. Dependency directories, generated build files, lockfile internals, Git history, and secret values are not reproduced. The file register explains the purpose of the project files without turning the report into a raw code dump.

A feature marked as implemented means its relevant code exists. It does not mean a live deployment, payment settlement, email delivery, or physical smart home connection was tested. Where comments or setup instructions make stronger claims than the implementation supports, the code is treated as the more reliable reference.

Contents

Chapter	What it covers
1	How to read this report
2	System architecture and technology
3	Customer storefront and navigation
4	Product discovery and shopping tools
5	Accounts, enquiries, and payment flow
6	Planning tools, projects, and consultations
7	Admin catalogue and operations
8	Admin content, access, and backups
9	Backend API and shared services
10	Data, synchronization, and integrations
11	Installation and demonstration guide
12	Implementation boundaries and next steps
13	Complete project file register

02 / PROJECT DOCUMENTATION

System architecture and technology

Three applications, distinct responsibilities

Application	Role	Default address
aetheria-smart-home	Customer storefront and smart home planning	localhost:3000
smarthomes-admin	Staff console for commerce and website operations	localhost:3001
smarthomes-backend	Shared REST API, JSON persistence, events, payments, email	localhost:5000

The storefront and admin console send HTTP requests to the backend. Their API helpers first consider VITE_API_URL. For a local machine or numeric LAN address they use port 5000 on the same hostname. For a public hostname they use the frontend origin, where Vite forwards /api requests to the local backend. This

supports the public tunnel demonstration path while avoiding a separate HTTP API URL in an HTTPS frontend session.

Main technology choices

Technology	Purpose in this folder
React 19 and TypeScript	Components, application state, typed records, and interactive forms.
Vite 6 and Tailwind CSS 4	Development servers, production frontend builds, and utility-based styling.
Lucide React and Motion	Interface icons and animation-related dependencies.
Node.js, Express 4, cors, dotenv	HTTP routes, cross-origin requests, and environment configuration.
Firebase SDK	Authentication calls, Firestore reads/writes, and live collection listeners.
Browser localStorage	Cached catalogue/content, cart, wishlist, comparison, sessions, and local fallback records.
Server-Sent Events	One-way backend notifications to connected browser clients.
Razorpay integration	Gateway order creation and signature checks, with a demonstration mode.
Nodemailer and jsPDF	Server HTML invoice email and browser-generated proposal PDFs.
Localtunnel	Temporary public links for the three development services.

Dependencies such as @google/genai are declared in the frontends, but a dependency declaration alone is not evidence of a customer-facing AI feature. The demonstrated product capabilities are those exposed by the source components and services.

Repository organisation

Each application owns its package manifest and configuration. The root package offers convenience commands for the three development services and dependency installation. Windows batch files launch the services in separate terminals. The storefront also contains an alternative Express server; the normal launcher uses the dedicated backend and Vite storefront instead.

Source: Root package.json; all three application package.json files; both vite.config.ts files; backend/src/server.ts; frontend API helpers.

03 / PROJECT DOCUMENTATION

Customer storefront and navigation

The storefront presents the project as a premium automation catalogue. Its visual system uses light backgrounds, restrained warm accents, serif headings, product imagery, and small uppercase labels. Shared navigation, footer, floating contact access, feedback messages, and shopping overlays keep the experience consistent across pages.

Page and route coverage

Area	Available screens
Discovery	Home, shop, categories, product details, solutions.
Planning and proof	Configurator, projects, individual project detail, consultation.
Customer workspace	Account, wishlist, cart, comparison, order enquiry.
Information and support	About, contact, FAQ, video library.
Policies	Privacy, shipping, terms, returns.
Fallback	Not-found screen for an unrecognised route.

App.tsx selects the page from activeRoute in ShopContext. The navigation state is based on the browser hash, while routeParams supplies details such as the selected product or project. This is a custom client-side navigation system. It should be checked when sharing deep links, because a hash route and an in-memory parameter are separate pieces of state.

Homepage composition

The home folder contains a hero banner, partner brands, product highlights, service explanation, applications, connected ecosystem, installation process, testimonials, reasons to choose the business, FAQ preview, and a smart room simulator. These components provide different kinds of reassurance: what is sold, where it can be used, how installation is approached, and what the experience might feel like.

The simulator changes visual room settings through browser state. Living room, bedroom, cinema, and terrace choices sit alongside lighting, brightness, climate, curtains, audio, security, doors, and display states. Scene presets such as Welcome Home apply coordinated values. These are interactive demonstrations; the inspected backend does not contain hardware control endpoints.

Shared interface details

The navigation includes product/category access and shopping actions. Breadcrumbs give page context. Cart and wishlist drawers offer short interactions without forcing a full page change. A quick-view modal gives a preview of a selected product. Toast messages confirm actions, empty-state components explain missing results, and the back-to-top control helps on longer pages. Floating WhatsApp access links customers to a contact channel.

Informational content

About, solutions, projects, policy, FAQ, video, and contact pages make the storefront useful beyond product transactions. The project pages draw on supplied portfolio data. The FAQ and video screens provide customer education, with the video page offering search and category browsing. Policy screens render the selected policy type rather than requiring four independent components.

Source: storefront/src/App.tsx; pages; components/layout; components/common; components/home; data/projectsData.ts and servicesData.ts.

04 / PROJECT DOCUMENTATION

Product discovery and shopping tools

Catalogue browsing

The shop is a searchable product catalogue with grid and list layouts. Customers can narrow results by category, protocol, and maximum price, then sort by newest, ascending price, descending price, or rating. Search matches product names, descriptions, and SKUs. A mobile filter state supports smaller screens, and an empty-result view gives feedback when a combination returns no products.

Price-on-request products bypass the normal maximum-price filter. Draft filtering in ShopPage checks the lowercase value draft. The backend uses statuses such as Active and Out of Stock, so status names need consistent normalisation when data moves between systems.

Product record and detail experience

Product records support identifiers, slugs, SKU, brand, category and subcategory, descriptions, price and original price, stock, status, featured flags, images, variants, specifications, feature lists, compatibility, applications, warranty, tags, catalogue page, and protocol. The backend normalises several legacy aliases so the frontends can consume a more consistent shape.

Product cards act as entry points to detail, quick view, wishlist, comparison, and enquiry actions. The detail page develops the selected product into a fuller presentation. Images, specifications, price information, and supporting product attributes make it possible to evaluate a device before contacting the business.

Cart

ShopContext maintains cart items as a product plus quantity. The storage service retains a compact saved representation and reconstructs the cart against the available catalogue. Customers can add products, update quantities, remove items, or clear the cart. Both the full cart page and drawer expose the shortlist; the enquiry modal can use the entire cart or a single selected product.

A cart is a browsing and request state. The backend stock reduction happens when an order record is created, not when an item enters the cart. The current implementation should therefore be demonstrated with a known stock level and a single controlled order.

Wishlist and comparison

Wishlist entries store product identifiers. Customers can retain products of interest and revisit them through a drawer or dedicated page. Comparison uses another set of saved identifiers and a separate screen to inspect selected products together. These lists are managed in the central shop state and browser storage, so persistence is tied to the browser and synchronization behaviour.

Seed catalogue

The backend seed data includes architectural switches and keypads, circadian smart lighting, multi-room audio, motorised shading, and biometric security categories. Example products include Lutron Palladiom and Crestron Horizon keypads, a Bang & Olufsen soundbar, and Somfy drapery hardware. These are sample catalogue records, not proof of certified integration or inventory held by an actual business.

Source: ShopPage.tsx; ProductDetailsPage.tsx; components/shop/ProductCard.tsx and ProductQuickView.tsx; context/ShopContext.tsx; backend/data/seedData.ts.

05 / PROJECT DOCUMENTATION

Accounts, enquiries, and payment flow

Customer registration and sign-in

The authentication modal has login, signup, and forgot-password tabs. AuthContext attempts Firebase sign-in or account creation, then uses StorageService to establish the local customer result. During registration it also mirrors name, email, and phone to the backend customer endpoint. Logout calls Firebase signOut and clears the local customer state.

The local fallback is a material part of the current implementation. A failed Firebase call is caught, and the local storage operation still runs. Consequently, the UI session should not be described as proof of a valid Firebase session. The account page displays the saved profile and filters stored enquiry history by the current email address; it is primarily a profile and history view.

Login before selected actions

requireAuth can hold a pending action and open the login modal. If the customer is already present, the callback runs immediately. If login or signup succeeds, the held callback resumes after a short delay. This creates a smoother flow for actions such as cart additions, wishlist changes, quote requests, and checkout when those controls use the guard.

Enquiry options

The shared enquiry modal provides quote, WhatsApp, email, call, and payment tabs. It derives line items either from the selected product or the cart, then calculates the total from each item price multiplied by quantity. Customer data can prefill the form. Quote submission creates a reference, records the enquiry, and clears the cart when the request came from the whole cart.

submitEnquiry first writes a local enquiry and returns that result. It then starts the backend order request in the background. Errors are logged rather than passed back to the immediate confirmation. A customer-facing reference therefore confirms the local write; staff should check that the matching backend record exists before treating the process as complete.

Payment sequence

Step	What the code does
1. Collect details	Require name, email, phone, and delivery address for the payment path.
2. Create gateway order	POST /api/payment/create-order converts the amount to paise.
3. Choose mode	Configured Razorpay keys invoke its API; missing keys return a simulated order.
4. Checkout or simulate	The modal loads the Razorpay SDK. Live mode opens checkout; demo mode creates synthetic payment identifiers.
5. Verify	POST /api/payment/verify compares an HMAC SHA-256 signature in the live path; simulated requests return success.
6. Record purchase	A successful result submits a purchase record and shows the reference/payment feedback.

Simulation does not transfer money. The code accepts a simulated flag at the verification endpoint, and the regular order endpoint can receive payment status directly from a client. These are demo conveniences and do not constitute a production payment authorization boundary.

A visible wording mismatch

OrderEnquiryPage says that no third-party gateway is integrated, while EnquiryModal includes Razorpay checkout and a payment tab. The report records both: the page copy is outdated relative to the payment code. The flow should be presented according to the modal behaviour actually being demonstrated.

Source: AuthContext.tsx; AuthPages.tsx; AccountPage.tsx; EnquiryModal.tsx; OrderEnquiryPage.tsx; ShopContext.tsx; backend/routes/payments.ts and orders.ts.

06 / PROJECT DOCUMENTATION

Planning tools, projects, and consultations

Guided smart home configurator

The configurator turns a broad automation request into structured selections. It walks through property type, property size, room counts, desired systems, and an automation tier before presenting results. Customers can choose a predefined size range or enter an exact square footage. Room counts include living spaces, bedrooms, kitchen, dining, bathrooms, office, cinema, outdoor space, garage, and other rooms.

System choices include smart lighting, motorised curtains, climate control, security, smart locks, multi-room audio, and whole-home automation. The tier choices are Essential, Luxury, and Architectural. Settings supply the available choices, unit costs, multipliers, and installation rate rather than placing every pricing value directly in the page.

How the estimate is calculated

Calculation	Rule used in ConfiguratorPage
Room cost	Sum of room count multiplied by the configured cost per unit.
System cost	Sum of each selected system cost per square foot multiplied by area.
Hardware subtotal	(Room cost + system cost) x property multiplier x tier multiplier.
Installation	Hardware subtotal x configured installation percentage / 100.
Indicative range	Lower bound is 90% and upper bound is 115% of subtotal plus installation, rounded to the nearest 1,000.

For example, if the combined pre-multiplier room and system cost were INR 100,000, both multipliers were 1, and installation were 18%, the total estimate would be INR 118,000. The displayed range would round to INR 106,000 through INR 136,000. This example explains the formula; it is not a quotation for a particular house.

A source comment mentions a different tolerance, but the implemented range is 90% to 115%. The result is a planning estimate. It does not replace a site survey, an agreed bill of materials, or a final installation quotation.

Proposal and enquiry output

The page creates a proposal record containing a generated reference, property details, rooms, selected systems, tier, and cost breakdown. The PDF service uses jsPDF to generate a downloadable proposal. A contact modal collects customer details for a detailed request, and the selected configuration can be sent through the common enquiry flow.

The generic backend order payload does not forward all configurator-specific fields. Some planning detail therefore exists in the local enquiry/proposal representation without being preserved as dedicated fields in the backend order. This is an important handover consideration for staff processing design requests.

Consultations and portfolio

The consultation page is the dedicated planning contact surface, with customer and property information, preferred appointment details, and request submission. The projects list and project-detail page show supplied installation examples. Solutions explains the types of automation offered. Together these pages support customers who need advice and a complete design rather than an individual product.

Source: ConfiguratorPage.tsx; ConsultationPage.tsx; ProjectsPage.tsx; ProjectDetailPage.tsx; SolutionsPage.tsx; services/pdfGenerator.ts; services/storage.ts; types.ts.

Admin catalogue and operations

The admin console separates staff work from the customer storefront. App.tsx chooses the active view from a sidebar menu, while the navbar shows the current context and session actions. On smaller screens the sidebar can be opened as a drawer. The console includes catalogue, sales, customer, content, access, and maintenance areas.

Dashboard

DashboardView loads operational statistics and recent records. Its state includes recent orders, recent products, low-stock entries, and activity logs. It also contains a Firestore write-test control and session reset action. These tools are useful for a demonstration and connection diagnosis; a dashboard statistic should be interpreted according to the records currently loaded into its cache.

Products and categories

ProductsView is the catalogue editor. Staff can create, edit, search, and remove products, maintain commercial fields and product descriptions, and control catalogue presentation. CategoriesView manages the category structure and related display information. Featured flags in product and category editors drive homepage selection; the featured-lists menu explains this and directs staff to the relevant editors.

The backend accepts product create/update requests and normalises IDs, slugs, SKU, images, price aliases, featured aliases, specifications, and defaults. Removing a product publishes a deletion event. Category records include ordering, active and featured flags, image, name, description, and timestamps.

Variants and inventory

VariantsView brings product variants into a management view and allows editing variant information. InventoryView represents both standalone product stock and variant stock, with inline stock editing and a configurable low-stock threshold. The same inventory component serves both the full inventory menu and the low-stock-only screen.

Variant stock exists in the admin view, but backend createOrder reduces the parent product stock using productId. It does not select and decrement a variant stock record. Demonstrations involving variants should not claim that the backend provides variant-level stock accounting.

Orders and customer ledger

OrdersView provides the staff-facing order workspace. The backend exposes order listing, single-record retrieval, and order/payment status changes. Customer records aggregate order count, purchase value, and last order date, and retain contact details and addresses. CustomersView gives staff a dedicated place to inspect and manage customer records.

The shared create-order implementation updates customer metrics for every record it creates, including records submitted through enquiry paths. Those aggregates should be described as current record totals rather than audited revenue or confirmed sales.

Payments

PaymentsView derives its list from order records. Staff can search and filter payment status, including Paid, Pending, and Failed, and manually mark a record as paid. This is a payment-status workspace. It is not a gateway settlement feed, refund processor, bank reconciliation service, or immutable financial ledger.

Source: admin/src/App.tsx; Sidebar.tsx; DashboardView.tsx; ProductsView.tsx; CategoriesView.tsx; VariantsView.tsx; InventoryView.tsx; OrdersView.tsx; CustomersView.tsx; PaymentsView.tsx; utils/db.ts.

08 / PROJECT DOCUMENTATION

Admin content, access, and backups

Website content management

Module	Editing capability
Homepage	Hero heading and description, CTA label/destination, banner image, promotional text/image/link.
FAQs	Manage question-and-answer content for customer support.
Videos	Title, teaser description, YouTube link, thumbnail, category, sort index, and display status.
Offers	Coupon code, title, description, percentage or fixed discount, scope/type, start/end dates, and active state.
SEO	Page title, description, keywords, canonical URL, Open Graph title/description/image, and index setting.
Settings	Maintain the system settings fields exposed by the settings form.

An editable record is only one half of content management. Its visible effect also depends on the storefront reading the same collection or settings document. The project has both backend CMS endpoints and Firestore-based content paths, so a saved change should be checked on the customer screen rather than assumed to appear everywhere.

Staff users and activity history

UsersView contains team-account creation, role changes, and deletion. Roles in the project include Super Admin and Catalogue Manager. The local session and Firestore authentication paths influence who reaches the console. ActivityLogView displays recorded actions; utility functions log changes such as offer creation or SEO updates.

The current log is an application activity aid. Local storage and editable cloud records do not make it a tamper-resistant audit system. Similarly, role labels in the interface do not prove that all backend operations enforce those roles.

Backup and cloud maintenance

BackupsView exports a JSON backup through a browser download. Import reads and parses JSON, stores the pending content, and displays a warning before restore. The restore helper handles the cached data and Firestore synchronization. This export should not be assumed to include the backend data/db.json file; that file is a separate persistence layer and needs its own backup.

The same view lets staff inspect and override Firebase connection settings, clear overrides, test a connection, seed cloud data, pull cloud records, and run security-oriented test controls. Those controls are useful diagnostics, but a green result from a selected test does not establish security across the whole system.

Admin session lock

The dedicated backend has lock-status, heartbeat, lock, and unlock routes. It retains an active session ID and last-seen time in memory, with a 45-second expiry. This supports a single active editing-session concept. The lock is lost on process restart and is not middleware protecting product, order, or CMS mutations.

Authentication fallback and bootstrap

LoginView includes permissive demonstration behaviour. When Firebase is unconfigured it can create a local admin session for an unknown email. Its cloud path can attempt account bootstrap, including an override account when an existing email cannot be signed in. The admin Firestore rules explicitly recognise override accounts. These conveniences must be removed or restricted before relying on the console for real staff access control.

Source: WebsiteView.tsx; FaqsView.tsx; VideosView.tsx; OffersView.tsx; SeoView.tsx; SettingsView.tsx; UsersView.tsx; ActivityLogView.tsx; BackupsView.tsx; LoginView.tsx; backend/routes/adminLock.ts; admin/firestore.rules.

09 / PROJECT DOCUMENTATION

Backend API and shared services

API surface

Method	Path	Purpose
GET	/api/health	Health, service version, timestamp, active SSE count.
GET	/api/events	Open the server event stream.
GET / POST	/api/products	Filter/list products; create or save a product.
GET / PUT / DELETE	/api/products/:id	Read, update, or remove one product.
GET / POST	/api/categories	Read categories; create/update a category.
DELETE	/api/categories/:id	Remove a category.
GET / POST	/api/orders	List records; create an order or enquiry.
GET	/api/orders/:id	Read an order by ID or order number.
GET	/api/orders/:id/invoice-html	Render the HTML invoice.
PATCH	/api/orders/:id/status	Change order or payment status.
GET	/api/orders/customers/all	Read customer ledger records.
POST	/api/orders/customers	Register or update a customer.
PATCH	/api/orders/customers/:id	Change customer fields.
POST	/api/payment/create-order	Create a live or simulated Razorpay order.
POST	/api/payment/verify	Verify a signature or accept the demo path.
GET / POST / DELETE	/api/cms/offers, /faqs, /videos	Read/save a record; DELETE uses /:id.
GET / POST	/api/cms/homepage, /settings, /seo	Read or save settings/content.
GET	/api/admin/lock-status	Check whether another session holds the lock.
POST	/api/admin/heartbeat, /lock, /unlock	Maintain or release the in-memory lock.

Server behaviour

server.ts enables JSON bodies up to 10 MB, development request logging, and broad CORS that echoes request origins with credentials enabled. The environment example contains CORS_ORIGIN, but the inspected server does not read it to build an origin allowlist. Product, order, category, and CMS routers do not use server authentication middleware.

Persistent JSON database

DatabaseService loads the shared database from data/db.json and uses seed data for an initial state. Mutations update the in-memory records and write a temporary JSON file before renaming it over the main file. The rename strategy reduces the chance of a partially written final file. It does not provide multi-process transactions, relational constraints, or durable message processing.

Order creation generates an ID and an ATH-style reference if none is supplied, stores customer and line-item information, derives subtotal/tax defaults, reduces product stock, updates customer metrics, saves the data, and broadcasts changes. Stock is clamped at zero; insufficient quantity is not rejected. Every line item with a matching product can reduce stock, regardless of the record being a paid purchase or an enquiry.

Event service

EventStreamManager retains connected HTTP responses. New clients receive a CONNECTED message. Mutations broadcast JSON messages containing type, data, and timestamp. The event types cover product changes, categories, orders, customers, offers, settings, and inventory. Clients can use these events to refresh their local cache and React state.

The code sends the initial handshake and mutation messages but does not include a recurring keepalive, replay history, or event IDs. It removes clients on close or write failure. Persistent proxies and reconnection scenarios need separate runtime checks. FAQ, video, and SEO persistence paths do not all broadcast a matching change event.

Invoice and email service

MailService builds an HTML invoice with customer/order context, item rows, quantities, unit prices, line totals, summary amounts, and business contact styling. The invoice endpoint returns that HTML to a browser. The create-order route starts email dispatch asynchronously, so an email problem does not directly block its response.

Transport selection prefers explicit SMTP settings, then Gmail settings, then an Ethereal test account. If test-account creation fails, the service can operate with a simulated/logging fallback. Gmail delivery requires valid credentials and network access. An Ethereal preview is not delivery to the customer inbox. The invoice code calculates an 18% tax estimate; the report does not certify tax compliance.

Source: backend/src/server.ts; all backend route files; services/db.ts; services/eventStream.ts; services/mailService.ts; types/index.ts.

Data, synchronization, and integrations

Three storage layers

Layer	Main role	Practical implication
Browser cache	Immediate UI state, saved lists, sessions, and fallback records.	Specific to browser/origin; clearing site data removes it.
Backend JSON	Shared local API catalogue, orders, customers, and CMS state.	Persistent on the server disk; separate from Firestore.
Firestore	Cloud collection/document synchronization and live listeners.	Requires correct project configuration, rules, schema, and connectivity.

The frontends use initial datasets to bootstrap or support fallback operation. Admin initDB hydrates multiple collections from the backend and creates an SSE listener. Firestore helpers mirror or pull records and subscribe to cloud changes. The storefront also attaches Firestore listeners and backend events. These routes improve responsiveness but create more than one possible source of truth.

The storefront and admin use different local storage keys. Each cache must be translated into its expected record shapes. Firestore collections include products, categories, offers, brands, FAQs, videos, enquiries, and settings-related documents. Not every backend field, frontend type, and rules validator has the same naming convention.

Main record families

Record	Information it represents
Product / variant	Catalogue identity, commercial data, stock, imagery, specifications, and selectable finishes or models.
Category	Navigation grouping, order, image, active status, and featured placement.
Order / enquiry	Customer identity, item lines, amount, address, status, request type, notes, and timestamps.
Customer	Contact details, addresses, account state, order count, aggregate value, last order date.
Offer	Promotion identity, discount mode/value, dates, scope, and active flag.
FAQ / video	Customer education text or media metadata with presentation fields.
Homepage / SEO / settings	Business/site content and page metadata.
Admin / activity	Staff identity and role/session information, plus recorded application actions.
Configurator / proposal	Property, area, rooms, selected systems, tier, rate settings, estimate, and proposal reference.

Firebase rules in the folder

Both frontend folders contain firestore.rules, and their policies are different. The storefront rules use validation helpers whose expected fields include category, stockQuantity, and stockStatus; backend products instead use fields such as categoryId, stock, and status. Rules and write payloads therefore need alignment. The admin rules include active admin documents, privileged roles, and override/initial-bootstrap conditions.

Rule files are configuration artifacts. Their presence does not prove which rules are currently deployed in Firebase. A correct assessment requires checking the target project and exercising its actual permissions, which was outside this document-generation review.

Public network access

tunnel.ts is the public-access helper used by the public launcher. It exposes the services through Localtunnel, while Vite handles API proxying. This allows a demonstration across separate networks when the tunnel service and host are reachable. Public links are temporary, and connectivity, gateway screens, timeouts, and firewalls can affect the experience. The existing setup guide overstates this as guaranteed access on any network.

Source: Frontend API/storage/Firebase helpers; admin/utils/db.ts; both firestore.rules files; backend/scripts/tunnel.ts; both vite.config.ts files.

11 / PROJECT DOCUMENTATION

Installation and demonstration guide

Prepare the project

Install a Node.js version compatible with the package requirements and confirm that node and npm are available. The repository guide mentions Node 18/20, but the exact supported runtime should be checked against the installed dependency versions. Run install-dependencies.bat or npm run install:all from the root to install packages for all three applications.

Prepare local environment files from the supplied .env.example files. Use real secrets only in local configuration, never in the report or a shared ZIP. The backend uses PORT and HOST, optional Razorpay keys, and SMTP or Gmail credentials. The frontend API override is VITE_API_URL. Firebase configuration is handled by the frontend Firebase helpers and supplied project configuration files.

Start locally

Task	Command or launcher
All local services	start-all.bat
Backend only	npm run dev:backend
Storefront only	npm run dev:storefront
Admin only	npm run dev:admin
Backend check	Open http://localhost:5000/api/health
Temporary public demonstration	start-with-public-internet.bat

The root npm run dev command starts only the backend. Use the batch launcher or the three named commands for the complete system. Keep the service terminals open. If a port is already occupied, close the conflicting process or update the relevant server and API settings consistently.

For another device on the same Wi-Fi, use the host computer LAN address with port 3000 for the storefront and 3001 for the admin console. The host must accept incoming connections and the network must permit device-to-device traffic. For separate networks, use the public launcher and the generated frontend HTTPS link while keeping the local processes running.

Email and payment preparation

For email, configure the transport supported by mailService.ts: SMTP_HOST, SMTP_PORT, SMTP_USER and SMTP_PASS (or its supported EMAIL aliases), or GMAIL_USER and GMAIL_APP_PASSWORD. Set SMTP_FROM and SUPPORT_PHONE as needed. Avoid leaving the example password text as a real configured value. The exact provider account requirements must be verified with the provider before a live demonstration.

For a demo payment, leave Razorpay keys blank and explicitly identify the result as simulated. For live operation, configure the gateway keys and validate the end-to-end order/verification flow in the provider environment. The simulation still attempts to load the checkout SDK before entering its demo branch, so the demo is not fully offline.

A practical demonstration sequence

1. Open the health endpoint, storefront, and admin console. Confirm that catalogue data loads in both interfaces.

2. Search the catalogue, apply a category/protocol filter, compare products, and add an item to the cart or wishlist.

3. Register or sign in as a customer. Submit one controlled quote and match its reference in the admin order view.

4. Demonstrate payment simulation if appropriate, identifying the synthetic payment identifiers and absence of a real money transfer.

5. Change a product in the admin console and check that the customer screen receives the update. Inspect the stream or refresh if synchronization fails.

6. Build a configurator estimate, adjust the area or tier, download the proposal, and submit a planning request.

7. Show inventory, customer records, homepage/FAQ/video editing, activity history, and backup export. Use sample data for restore or cloud seeding demonstrations.

Troubleshooting

Symptom	Where to look
No catalogue or API errors	Backend terminal, /api/health, VITE_API_URL, proxy, and browser network requests.
Changes appear in one screen only	SSE connection, Firestore project/rules, cache keys, and record field compatibility.
Login works locally but cloud calls fail	Auth fallback behaviour and actual Firebase authentication state.
No customer email	Transport settings and backend logs; distinguish Gmail delivery from Ethereal preview.
Public link fails	Tunnel process, local service ports, provider gateway, and host/network connectivity.
Proposal download fails	Browser console, jsPDF service, required form data, and browser download handling.

Source: README.md; SETUP.md; root package.json; launch/install scripts; .env.example files; API helpers; mailService.ts.

12 / PROJECT DOCUMENTATION

Implementation boundaries and next steps

The folder provides a broad project demonstration: product discovery, planning, customer requests, operational administration, shared records, events, payment integration code, and invoice output. Its most useful next step is to make the existing paths consistent and reliable before adding more screens.

What is a simulation or depends on configuration

Capability	Current interpretation
Smart room scenes	Browser UI simulation; no inspected physical device control service.
Payments without keys	Simulated gateway order and verification; no settlement.
Live gateway checkout	Integration code exists; requires credentials and provider validation.
Email	HTML invoice generation exists; delivery depends on a configured transport and connectivity.
Customer authentication	Firebase attempted with a local result/fallback afterward.
Admin authentication	Local fallback and automatic cloud bootstrap/override paths exist.
Cloud synchronization	Code and rules are present; deployed permissions and field compatibility are unverified.
Public access	Temporary tunnel workflow exists; network availability is conditional.
SEO settings	Editor and records exist; page metadata consumption must be checked.

Changes that would strengthen the current system

First, establish one authoritative source for catalogue, order, and content records. Align frontend types, API payloads, and Firestore validators. Preserve consultation/configurator detail in the backend, and return a confirmed server result before displaying a completed submission.

Second, enforce access on the server for staff operations and customer-specific data. Remove permissive login bootstrap and override paths from normal operation. Replace local password/session assumptions with a verified identity flow and restrict CORS to intended applications. The current session lock does not replace authentication.

Third, calculate trusted totals on the server from stored catalogue prices, separate quotations from purchases, validate quantities and stock, bind payment verification to the matching order and amount, and prevent duplicate submissions. Add variant-aware inventory handling and explicit cancellation/restock rules.

Fourth, make synchronization predictable. Add event keepalives and recovery handling where required, broadcast missing content updates, and show failed cloud/API writes in the interface. Back up both the backend JSON database and cloud data, with clear restore scope.

Finally, improve the deployment handover. Replace outdated checkout copy, validate invoice totals and business details, define a compatible runtime, and run focused integration checks. The backend seed script command points to src/scripts/seed.ts, but that script was not present in the inspected project file list; initialization instead exists in the database/seed-data path.

Suggested verification checklist

Test	Evidence to collect
Order flow	Matching server reference, correct items/amount, expected stock change, and customer ledger update.
Quote flow	Pending payment state, complete request detail, and intended stock behaviour.
Payment flow	Demo labelled clearly; live signature failure rejected; correct order binding and duplicate protection.
Access control	Unauthorized staff mutations and unrelated customer reads rejected by the actual server/cloud.
Synchronization	Admin edit visible in storefront; refresh/reconnect recovers missed state.
Proposal / invoice	Legible generated output and reconciled amounts.
Recovery	Export/restore scope documented; backend disk data recoverable separately.

These are proposed verification tasks. This PDF was checked for document readability and coverage, but the applications were not launched, built, or tested against live Firebase, email, Razorpay, or hardware during this review.

Source: Findings from the inspected source files; no live service or runtime certification is implied.

13 / PROJECT DOCUMENTATION

Complete project file register

This register covers the first-party files found in the project scan, grouped by responsibility. It includes application source, setup material, and supporting configuration. Installed dependency trees, generated build output, secret .env values, and Git internals are intentionally outside the register.

Root

File within this folder	Responsibility
install-dependencies.bat	Installs dependencies for the three application folders.
package.json	Application dependencies and runnable scripts; the root version coordinates subprojects.
README.md	Project introduction, default service addresses, and setup pointers.
SETUP.md	Installation, local/public access, email setup, and presentation/viva guide; some claims require qualification.
start-all.bat	Windows launcher for the backend, storefront, and admin development services.
start-with-public-internet.bat	Windows launcher adding temporary public tunnels to the three services.

aetheria-smart-home

File within this folder	Responsibility
.env.example	Example environment configuration; use as a setup template, not a source of real secrets.
bun.lock	Bun package-manager lockfile retained with the project.
firebase-applet-config.json	Firebase/applet project configuration artifact.
firebase-blueprint.json	Firebase integration blueprint/configuration artifact.
firestore.rules	Cloud database access rules; deployment and schema alignment must be verified.
index.html	Frontend HTML entry shell.
metadata.json	Application metadata retained with the frontend project.
package-lock.json	npm dependency resolution lockfile for reproducible installs.
package.json	Application dependencies and runnable scripts; the root version coordinates subprojects.
README.md	Project introduction, default service addresses, and setup pointers.
server.ts	Express service entry; the storefront copy is an alternative server path.
src/App.tsx	Top-level page/view selection and application composition.
src/components/common/Breadcrumbs.tsx	Page hierarchy/context display.
src/components/common/EmptyState.tsx	Reusable missing-content/results feedback.
src/components/common/RatingStars.tsx	Product rating visual display.
src/components/common/ToastContainer.tsx	Transient action feedback display.
src/components/common/ValueTrustBadges.tsx	Reusable service/trust messaging.
src/components/home/ApplicationsSection.tsx	Automation application/use-case presentation.
src/components/home/ConnectedEcosystemSection.tsx	Connected-platform/ecosystem presentation.
src/components/home/FaqPreviewSection.tsx	Homepage FAQ preview.
src/components/home/HeroBanner.tsx	Landing hero and primary call to action.

File within this folder	Responsibility
src/components/home/InstallationProcess.tsx	Installation journey presentation.
src/components/home/PartnerBrands.tsx	Partner/brand presentation.
src/components/home/ProductHighlights.tsx	Highlighted catalogue products.
src/components/home/SmartHomePackages.tsx	Supporting project artifact.
src/components/home/SmartRoomSimulator.tsx	Interactive browser room/scene demonstration.
src/components/home/TestimonialsSection.tsx	Customer testimonial presentation.
src/components/home/WhatWeDoSection.tsx	Automation service introduction.
src/components/home/WhyChooseUs.tsx	Business/service value presentation.
src/components/layout/BackToTop.tsx	Long-page scroll return control.
src/components/layout/CategoryDropdown.tsx	Navigation category chooser.
src/components/layout/FloatingWhatsApp.tsx	Floating customer contact link.
src/components/layout/Footer.tsx	Storefront footer navigation and business information.
src/components/layout/Navbar.tsx	Shared navigation header for the containing interface.
src/components/shop/CartDrawer.tsx	Compact cart overlay.
src/components/shop/EnquiryModal.tsx	Quote/contact/payment modal and checkout interaction.
src/components/shop/ProductCard.tsx	Catalogue product tile and action entry points.
src/components/shop/ProductQuickView.tsx	Product preview modal.
src/components/shop/WishlistDrawer.tsx	Compact saved-product overlay.
src/context/AuthContext.tsx	Customer identity state, auth modal controls, and pending-action guard.
src/context/ShopContext.tsx	Catalogue, navigation, cart, saved lists, enquiries, and toast state.
src/data/catalogueData.ts	Storefront sample product/category data.
src/data/projectsData.ts	Storefront project/portfolio content.
src/data/servicesData.ts	Storefront service/solution content.
src/index.css	Global styles and presentation foundation.

File within this folder	Responsibility
src/lib/firebase.ts	Firebase app/auth/database setup for the containing application.
src/main.tsx	React entry point that mounts the app.
src/pages/AboutPage.tsx	Business/brand information.
src/pages/AccountPage.tsx	Customer profile and email-matched enquiry history.
src/pages/AuthPages.tsx	Customer login, registration, and recovery-tab interface.
src/pages/CartPage.tsx	Cart quantities, totals, and enquiry entry.
src/pages/CategoriesPage.tsx	Category organisation and editing.
src/pages/ComparePage.tsx	Selected-product comparison screen.
src/pages/ConfiguratorPage.tsx	Guided property/system selection and estimate/proposal output.
src/pages/ConsultationPage.tsx	Property/customer/appointment enquiry interface.
src/pages/ContactPage.tsx	Business contact information and customer contact surface.
src/pages/FAQPage.tsx	Customer questions and answers.
src/pages/HomePage.tsx	Composed customer landing page.
src/pages/NotFoundPage.tsx	Fallback for unknown customer routes.
src/pages/OrderEnquiryPage.tsx	Enquiry checkout landing page; gateway wording is outdated.
src/pages/PolicyPage.tsx	Privacy, shipping, terms, and returns content renderer.
src/pages/ProductDetailsPage.tsx	Selected product presentation and shopping/contact actions.
src/pages/ProjectDetailPage.tsx	Individual portfolio/project view.
src/pages/ProjectsPage.tsx	Portfolio/project listing.
src/pages/ShopPage.tsx	Search, filters, sorting, and catalogue layouts.
src/pages/SolutionsPage.tsx	Automation solution/service presentation.
src/pages/VideosPage.tsx	Video-library metadata editing.
src/pages/WishlistPage.tsx	Saved-product screen.
src/services/api.ts	Backend URL resolution, API requests, and event subscription helpers.

File within this folder	Responsibility
src/services/firebaseService.ts	Storefront cloud seeding and live collection/settings synchronization.
src/services/pdfGenerator.ts	jsPDF smart home proposal generation and currency formatting.
src/services/storage.ts	Browser persistence helpers, local records, and default content/configurator settings.
src/types.ts	Frontend record interfaces and shared type contracts.
tsconfig.json	TypeScript compiler and module configuration.
vite.config.ts	Vite plugins, aliases, frontend port, host binding, and development API proxy.

smarthomes-admin

File within this folder	Responsibility
.env.example	Example environment configuration; use as a setup template, not a source of real secrets.
app/applet/run_test.ts	Applet test utility retained under the admin applet folder.
bun.lock	Bun package-manager lockfile retained with the project.
firebase-applet-config.json	Firebase/applet project configuration artifact.
firestore.rules	Cloud database access rules; deployment and schema alignment must be verified.
index.html	Frontend HTML entry shell.
metadata.json	Application metadata retained with the frontend project.
package-lock.json	npm dependency resolution lockfile for reproducible installs.
package.json	Application dependencies and runnable scripts; the root version coordinates subprojects.
README.md	Project introduction, default service addresses, and setup pointers.
src/App.tsx	Top-level page/view selection and application composition.
src/components/ActivityLogView.tsx	Recorded application actions.
src/components/BackupsView.tsx	JSON backup/restore and Firebase maintenance tools.
src/components/CategoriesView.tsx	Category organisation and editing.
src/components/CustomersView.tsx	Customer records and account management.
src/components/DashboardView.tsx	Statistics, recent operational records, and diagnostic controls.
src/components/FaqsView.tsx	FAQ content editing.
src/components/InventoryView.tsx	Stock editing and full/low-stock views.
src/components/LoginView.tsx	Admin login and fallback/bootstrap session handling.
src/components/Navbar.tsx	Shared navigation header for the containing interface.
src/components/OffersView.tsx	Coupon/promotion dates and discount editing.

File within this folder	Responsibility
src/components/OrdersView.tsx	Order/request inspection and status management.
src/components/PaymentsView.tsx	Order-derived payment list, filters, and manual status action.
src/components/ProductsView.tsx	Catalogue list and product editing.
src/components/SeoView.tsx	Page metadata and Open Graph editor.
src/components/SettingsView.tsx	System settings form.
src/components/Sidebar.tsx	Admin menu grouping and responsive drawer.
src/components/UsersView.tsx	Staff accounts and roles.
src/components/VariantsView.tsx	Variant list and editing across products.
src/components/VideosView.tsx	Video-library metadata editing.
src/components/WebsiteView.tsx	Homepage hero, promotion, and CTA editing.
src/data/initialData.ts	Admin initial catalogue, users, orders, settings, and content records.
src/index.css	Global styles and presentation foundation.
src/main.tsx	React entry point that mounts the app.
src/types.ts	Frontend record interfaces and shared type contracts.
src/utils/api.ts	Backend URL resolution, API requests, and event subscription helpers.
src/utils/db.ts	Admin cache/cloud/backend helpers or backend JSON database service, according to folder.
src/utils/firebase.ts	Firebase app/auth/database setup for the containing application.
tsconfig.json	TypeScript compiler and module configuration.
vite.config.ts	Vite plugins, aliases, frontend port, host binding, and development API proxy.

smarthomes-backend

File within this folder	Responsibility
.env.example	Example environment configuration; use as a setup template, not a source of real secrets.
data/db.json	Persisted backend database snapshot; values may change during operation.
package-lock.json	npm dependency resolution lockfile for reproducible installs.
package.json	Application dependencies and runnable scripts; the root version coordinates subprojects.
src/data/seedData.ts	Backend initial catalogue and content records.
src/routes/adminLock.ts	In-memory active-session lock and heartbeat routes.
src/routes/categories.ts	Category API routes.
src/routes/cms.ts	Offers, FAQs, videos, homepage, system settings, and SEO API routes.
src/routes/orders.ts	Order, invoice, customer registration, and status API routes.
src/routes/payments.ts	Gateway order creation and live/demo payment verification routes.
src/routes/products.ts	Product API routes.
src/scripts/tunnel.ts	Temporary public tunnel orchestration.
src/server.ts	Express service entry; the storefront copy is an alternative server path.
src/services/db.ts	Admin cache/cloud/backend helpers or backend JSON database service, according to folder.
src/services/eventStream.ts	Connected-client registry and SSE broadcasting.
src/services/mailService.ts	Invoice HTML generation and SMTP/Gmail/test transport dispatch.
src/types/index.ts	Backend shared record/type definitions.
src/types/localtunnel.d.ts	Local TypeScript declaration for the tunnel package.
tsconfig.json	TypeScript compiler and module configuration.

The register contains 140 first-party project files. All descriptions are tied to the containing application; similarly named files in separate apps serve different runtime contexts.

Dependency and generated folders

node_modules contains installed third-party packages and can be recreated through dependency installation. dist/build folders, when present, contain generated output. package-lock.json and bun.lock record dependency resolution. These supporting items belong to the project environment but are not independent product features.

Source note

Source references throughout the report are paths within the supplied project folder. No external facts, provider instructions, or certification claims are added. The report is a source-based explanation of this snapshot, dated 11 October 2026.